

# LAB TASK 12

## TASK 1

### CODE

```
#include<iostream>
#include<string>
using namespace std;
class Order{
private:
    string itemName;
    int price;
    int quantity;
    string custName;
    int priority;
public:
    Order(string itName = "", int p = 0,int q = 0, string cName = "",int prior =0) {
        itemName = itName;
        price = p;
        quantity = q;
        custName = cName;
        priority = prior;
    }
    void getValues() {
        cout<<"Enter item name"<<endl;
        cin.ignore();
        getline(cin , itemName);
        cout<<"Enter price"<<endl;
        cin >> price;
        cout<<"Enter quantity"<<endl;
        cin>> quantity;
        cout<<"Enter customer name"<<endl;
        cin.ignore();
        getline(cin , custName);
        cout<<"Enter priority"<<endl;
        cin >> priority;
    }

    void display() {
        cout<<"Customer: "<<custName<<" | Item: "<<itemName<<" | Quantity: "<<quantity<<"
        Price: $"<<price<<" | priority: "<<priority<<endl;
    }

    int getPriority() const {
        return priority;
    }

    bool isEmpty() const {
        return itemName.empty();
    }
};
```

```

class MaxHeap {
private:
    Order* heap;
    int capacity;
    int size;

    int getParent(int i) {
        if(i == 0) return -1;
        return (i - 1) / 2;
    }

    int getLeft(int i) {
        int left = 2 * i + 1;
        return (left < size) ? left : -1;
    }

    int getRight(int i) {
        int right = 2 * i + 2;
        return (right < size) ? right : -1;
    }

    void swap(Order &a, Order &b) {
        Order temp = a;
        a = b;
        b = temp;
    }

    void heapifyUp(int i) {
        if(i <= 0) return;
        int parent = getParent(i);
        if(parent >= 0 && heap[i].getPriority() > heap[parent].getPriority()) {
            swap(heap[i], heap[parent]);
            heapifyUp(parent);
        }
    }

    void heapifyDown(int i) {
        int largest = i;
        int left = getLeft(i);
        int right = getRight(i);

        if(left != -1 && heap[left].getPriority() > heap[largest].getPriority()) largest = left;

        if(right != -1 && heap[right].getPriority() > heap[largest].getPriority()) largest = right;

        if(largest != i) {
            swap(heap[i], heap[largest]);
            heapifyDown(largest);
        }
    }
public:
    bool isEmpty() {return size == 0;}

```

```

MaxHeap(int cap = 100) {
    capacity = cap;
    size = 0;
    heap = new Order[capacity];
}

void insert(Order order) {
    if(size >= capacity) {
        cout<<"Heap is full.. Cannot take new orders"<<endl;
        return;
    }

    heap[size] = order;
    size++;

    heapifyUp(size - 1);

    cout<<"Order inserted successfully"<<endl;
}

void serverOrder() {
    if(size == 0) {
        cout<<"No order to serve"<<endl;
        return;
    }
    cout<<"---- Serving Order ----"<<endl;
    heap[0].display();

    heap[0] = heap[size - 1];
    size--;

    if(size > 0) heapifyDown(0);

    cout<<"Order served successfully"<<endl;
}

void displayAll() {
    if (size == 0) {
        cout<<"No orders in the system!"<<endl;
        return;
    }
    for (int i = 0; i < size; i++) heap[i].display();
    cout<<endl;
}
};

int main() {
    MaxHeap max(100);
    int choice;

    do {
        cout<<"1. Insert New Order"<<endl;
        cout<<"2. Serve Order"<<endl;
        cout<<"3. Display Orders"<<endl;
    }
}

```

```
cout<<"0. Exit"<<endl;
cout<<"Enter your choice: ";
cin >> choice;

switch(choice) {
    case 1: {
        Order newOrder;
        newOrder.getValues();
        max.insert(newOrder);
    }
    break;

    case 2:
        max.serverOrder();
    break;

    case 3:
        max.displayAll();
    break;

    case 4:
        cout<<"Exiting...."<<endl;
    break;

    default:
        cout<<"Invalid choice"<<endl;
    break;
}
} while (choice != 0);

return 0;
}
```